

The Bash Tool: Configuration, Security, and nice/ionice Priority Management

The `bash` tool is the workhorse of `ragent`. It executes shell commands from the agent's working directory, applies a seven-layer security model, and by default runs every command at a deliberately **lowered CPU and I/O priority** so that heavy agent workloads do not make the host machine unresponsive.

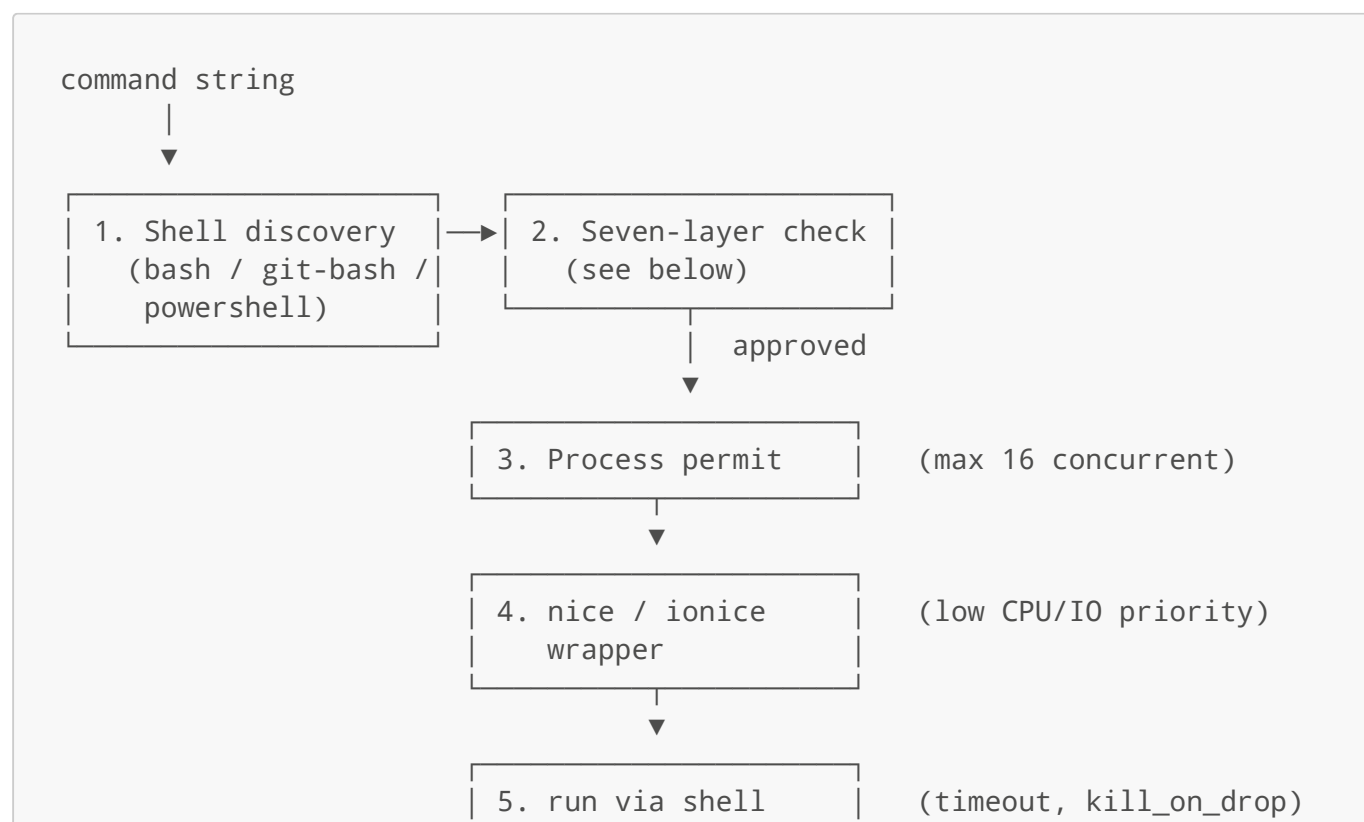
This document covers:

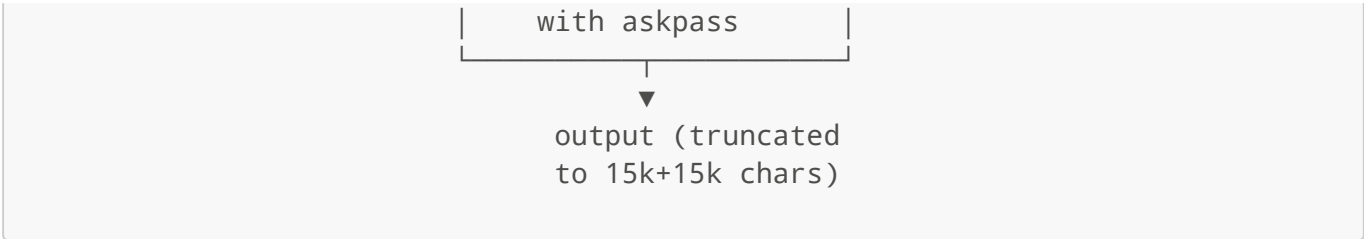
1. [Overview](#)
2. [Configuration](#) — the `bash` block in `ragent.json`
3. [The seven-layer security model](#)
4. [nice / ionice priority management](#)
5. [The `/bash` slash commands](#)
6. [Runtime behaviour](#) — timeout, output truncation, process limits
7. [YOLO mode](#)
8. [Platform notes](#)
9. [Reference](#)

Overview

The tool is implemented in `crates/ragent-tools-core/src/bash.rs` (~1500 lines) as the `BashTool` struct. It is a unit struct with no fields of its own — all behaviour is driven by the global `ragent_config` at execution time, populated from your config files.

Every command an agent runs through `bash` goes through the same pipeline:





Configuration

The bash tool is configured via the top-level `bash` block in `ragent.json` (global at `~/.config/ragent/ragent.json`, or project at `.ragent/ragent.json`).

The `bash` config block

```
{
  "bash": {
    "allowlist": ["curl", "wget"],
    "denylist": ["git push --force", "systemctl disable"],
    "nice": 10
  }
}
```

Key	Type	Default	Purpose
<code>allowlist</code>	<code>string[]</code>	<code>[]</code>	Command prefixes exempted from the built-in banned-command check
<code>denylist</code>	<code>string[]</code>	<code>[]</code>	Substring patterns that unconditionally reject a command
<code>nice</code>	<code>integer</code> or <code>null</code>	<code>10</code>	CPU niceness applied to every shell command (<code>-20 ... 19</code>); <code>null</code> disables priority wrapping

The underlying Rust struct (`crates/ragent-config/src/config.rs`, lines 1076–1113):

```
pub struct BashConfig {
    /// Command prefixes exempted from the banned-command check.
    pub allowlist: Vec<String>,
    /// Patterns that unconditionally reject a command.
    pub denylist: Vec<String>,
    /// Run shell commands at low CPU/IO priority (`nice -n <nice_level>`
and,
    /// on Linux, `ionice -c 3`) so that heavy agent workloads do not make
the
    /// host system unresponsive.
    pub nice: Option<i32>,
}
```

The `nice` field defaults to `Some(10)` via `default_nice_level()`:

```
const fn default_nice_level() -> Option<i32> {
    Some(10)
}
```

Merge semantics across global + project config

When both a global and a project config exist, ragent merges them:

- **allowlist** and **denylist** entries are **unioned** (deduplicated). An entry added in either config applies everywhere.
- The **nice** field is **not merged** from the project overlay — the base (global) value, or the **10** default, wins. Set it explicitly in the global config to change it.

Config flow

The bash tool does **not** read `Config` directly. At startup (and on `/reload`) ragent copies the merged config into an in-memory snapshot via `BashLists::load_from_config()` (`crates/ragent-config/src/bash_lists.rs`):

- TUI startup: `crates/ragent-tui/src/app/init.rs:397`
- `/reload`: `crates/ragent-tui/src/app/slash.rs:3361`

```
pub fn load_from_config() {
    let lists = match crate::config::Config::load() {
        Ok(cfg) => BashLists {
            allowlist: cfg.bash.allowlist,
            denylist: cfg.bash.denylist,
            nice: cfg.bash.nice,
        },
        ...
    };
}
```

The tool reads from this snapshot at execution time through helpers such as `nice_level()`, `is_allowlisted()`, and `matches_denylist()`.

The seven-layer security model

Every command is validated by seven layers, in order. The table below summarises them; full detail follows.

Layer	Check	Source	Bypassed by YOLO	Bypassed by allowlist
1	Safe-command whitelist	<code>SAFE_COMMANDS</code> const	—	—

Layer	Check	Source	Bypassed by YOLO	Bypassed by allowlist
2	Banned commands	<code>BANNED_COMMANDS</code> const	yes	yes
3	Denied commands & patterns	<code>DENIED_COMMANDS</code> , <code>DENIED_COMMAND_PATTERNS</code> , <code>DENIED_PATTERNS</code> consts	yes	no
4	Directory-escape	<code>is_directory_escape_attempt()</code>	—	—
5	Syntax validation	<code>validate_bash_syntax()</code>	—	—
6	User denylist	<code>matches_denylist()</code>	yes	no
7	Obfuscation detection	<code>validate_no_obfuscation()</code>	yes	no

Layer 1 — Safe-command whitelist

A built-in list of ~50 commands is prefix-matched and auto-approved (it just logs "`Safe bash command auto-approved`"). A command is safe if it **equals** an entry or **starts with the entry followed by a space**, so `ls` matches `ls -la` and `git` matches `git status`.

```
const SAFE_COMMANDS: &[&str] = &[
    // File management
    "ls", "cd", "pwd", "mkdir", "touch", "cp", "mv",
    // File reading & search
    "cat", "head", "tail", "grep", "egrep", "fgrep", "find", "rg", "wc",
    // Version control
    "git", "gh",
    // Build / package management
    "cargo", "rustc", "rustfmt", "clippy-driver", "npm", "yarn", "pnpm",
    "node", "npx", "python3", "python", "pip", "pip3", "make", "docker-
compose",
    // Text / data utilities
    "echo", "printf", "chmod", "jq", "yq", "sed", "awk", "sort", "uniq",
    "cut", "tr", "xargs", "date", "which", "tree", "diff", "patch",
];
```

Note: `rm` is deliberately excluded from the whitelist so it always requires a permission decision (and is caught by Layer 3 for destructive forms).

Layer 2 — Banned commands

A word-boundary match against `BANNED_COMMANDS`. These are network / exfiltration and attack tools that are almost never legitimate inside an agent:

```
const BANNED_COMMANDS: &[&str] = &[
    // Network / data-exfiltration tools
    "curl", "wget", "nc", "netcat", "telnet", "axel", "aria2c", "lynx",
    "w3m",
    // Attack and exploitation tools
    "nmap", "masscan", "nikto", "sqlmap", "hydra", "john", "hashcat",
    "aircrack", "metasploit", "msfconsole", "msfvenom", "burpsuite",
    "ettercap", "arp spoof",
    // Packet capture (enable via YOLO)
    "tcpdump", "wireshark",
];
```

A banned command is rejected **unless**:

- YOLO mode is enabled, **or**
- the command's **first token** is on the user **allowlist** (this is how you re-enable **curl** / **wget** without turning on YOLO).

```
if contains_banned_command(command) {
    if ragent_config::yolo::is_enabled() { /* allow */ }
    else if ragent_config::bash_lists::is_allowlisted(command) { /* allow
*/ }
    else { bail!("Command rejected: uses banned external tool (curl, wget,
nc, ...)); }
}
```

Layer 3 — Denied commands & patterns

Three sub-checks that are **never** bypassed by the allowlist (only by YOLO):

3a. Denied commands (word-boundary):

```
const DENIED_COMMANDS: &[&str] = &[
    "mkfs", "wipefs", // Disk / partition destruction
    "insmod", // Kernel modifications
    "useradd", "usermod", "groupadd", // User/group manipulation
    "visudo", // System configuration
    "grub-install", "efibootmgr", // Boot/firmware
];
```

3b. Denied command patterns (prefix patterns):

```
const DENIED_COMMAND_PATTERNS: &[&str] = &[
    "sudo ", "sudo\t", "su -", "su root", "doas ", // privilege escalation
    "passwd ", // user/group
    manipulation
];
```

```
"crontab -", // system configuration
];
```

3c. Denied substring patterns — the most destructive forms:

```
const DENIED_PATTERNS: &[&str] = &[
    // Destructive filesystem operations
    "rm -rf /", "rm -r -f /", "rm -fr /", "rm -Rf /", "rmdir /",
    // Disk operations
    "dd if=", "shred /dev",
    // Device writes
    "> /dev/sd", "> /dev/nvme", "> /dev/vd",
    // Fork bomb
    ":{(){ :|:&};:;",
    // Privilege escalation
    "chmod -R 777 /", "chown -R",
    // Network exfiltration of sensitive files
    "curl.*etc/shadow", "wget.*etc/shadow",
    // History / credential file theft
    ".bash_history", ".ssh/id_",
    // Kernel modifications
    "modprobe -r", "sysctl -w",
    // Destructive git operations
    "git push --force", "git push -f ", "git push origin --delete",
    // More destructive patterns
    "rm -rf ~", "rm -rf $HOME", "rm -rf .",
    "chmod 000 /", "chmod -R 000",
    // Data exfiltration via pipes
    "> /dev/tcp", "bash -i >&", "/dev/tcp/", "/dev/udp/",
    "systemctl disable", "systemctl mask", "chattr +i",
];
```

Layer 4 — Directory-escape prevention

`is_directory_escape_attempt(command, &ctx.working_dir)` rejects commands that `cd` or `pushd` out of the working directory via `..`, `~`, `$HOME`, `${HOME}`, or absolute `/` paths (and, on Windows, `C:\` / `\` drive roots). This keeps the agent's filesystem view inside its sandbox. This layer is **never** overridden.

Layer 5 — Syntax validation

`validate_bash_syntax()` runs the command through the shell's own parser with a **1-second timeout** and `kill_on_drop(true)`:

```
bash -n -c "<command>"
```

A non-zero exit produces `Bash syntax error: <stderr>` and the command is rejected. This uses the actual discovered shell (not a hardcoded `sh`) so it works on Git Bash too. Syntax validation is **skipped for**

PowerShell (which has its own runtime parser and no `-n` equivalent). This layer is never overridden.

Layer 6 — User denylist

Substring-matches the command against your configured `denylist` patterns. This is where project-specific guardrails go. Bypassed in YOLO mode only.

```
if !ragent_config::yolo::is_enabled()
  && let Some(pattern) =
    ragent_config::bash_lists::matches_denylist(command)
{ bail!("matches user-defined deny pattern ..."); }
```

Important asymmetry: an allowlist entry only exempts a command from **Layer 2 (banned commands)**. It does **not** exempt it from the denylist (Layer 6) or the denied-pattern checks (Layer 3). That is by design.

Layer 7 — Obfuscation detection

`validate_no_obfuscation()` rejects clearly-obfuscated payloads:

- `base64 ... | bash / | sh`
- `python / perl` using `exec( / eval(`
- `$(\xNN` hex-escape sequences
- `eval` combined with `$(...)` command substitution

Bypassed in YOLO mode only.

nice / ionice priority management

This is the heart of the question "how does the bash tool use nice/ionice". By default, **every** shell command `ragent` executes runs at a lowered CPU and I/O priority so that long agent jobs (compiles, tests, large grep runs) do not starve the interactive host.

The wrapper

When a POSIX shell is used on a non-Windows host, and `nice` is configured (default `10`), the command is prepended with a low-priority argv prefix:

- **Linux:** `nice -n 10 ionice -c 3`
- **Other Unix (macOS/BSD):** `nice -n 10`
- **Windows (Git Bash / PowerShell):** no prefix (see [Platform notes](#))

So a command like `cargo build` is actually executed as:

```
nice -n 10 ionice -c 3 bash -c '<wrapper>'
```

- `nice -n 10` lowers the **CPU scheduling priority** of the command and all its children (niceness `10` is clearly below normal priority `0`, but not so low that the shell becomes sluggish).
- `ionice -c 3` sets the I/O class to **idle** — the process only touches the disk when no other process needs it, preventing heavy I/O from blocking interactive work.

Where the prefix is built

`low_priority_prefix()` in `crates/ragent-tools-core/src/bash.rs` (lines 1028–1048):

```
fn low_priority_prefix(shell: &ShellType) -> Vec<std::ffi::OsString> {
    // POSIX wrappers only; Git Bash on Windows also understands them but
    the
    // shell executable may not be a standard `nice`/`ionice`, so restrict
    the
    // wrappers to native POSIX shells.
    if matches!(shell, ShellType::PowerShell(_)) || is_windows() {
        return Vec::new();
    }

    let Some(level) = ragent_config::bash_lists::nice_level() else {
        return Vec::new();
    };

    let mut prefix: Vec<std::ffi::OsString> =
        vec!["nice".into(), "-n".into(), level.to_string().into()];
    if cfg!(target_os = "linux") {
        prefix.push("ionice".into());
        prefix.push("-c".into());
        prefix.push("3".into());
    }
    prefix
}
```

If `nice` is set to `null` in config, `nice_level()` returns `None` and no prefix is added — commands run at **normal priority**.

Where the prefix is applied

`prepend_low_priority()` (lines 1058–1075) rewrites the `tokio::process::Command` so the original program becomes a **child** of the `nice ... ionice ...` prefix, carefully carrying over `kill_on_drop`:

```
fn prepend_low_priority(cmd: &mut Command, shell: &ShellType) {
    let prefix = low_priority_prefix(shell);
    if prefix.is_empty() { return; }

    let original_program = cmd.as_std().get_program().to_os_string();
    let original_args: Vec<std::ffi::OsString> =
cmd.as_std().get_args().map(Into::into).collect();
    let kill_on_drop = cmd.get_kill_on_drop();
```



```
let mut rebuilt = Command::new(prefix.first().unwrap());
rebuilt.args(&prefix[1..]);
rebuilt.args([original_program]);
rebuilt.args(original_args);
rebuilt.kill_on_drop(kill_on_drop);

*cmd = rebuilt;
}
```

It is applied to **all four** process-spawn paths:

Path	Call site (bash.rs)
Foreground bash in execute()	line 1329
Foreground Git Bash in execute()	line 1345
Foreground PowerShell in execute()	line 1365
Background shell (spawn_background_shell)	lines 1101, 1113, 1128

So **background commands launched via the **bg** tool receive the same low-priority treatment** as foreground commands.

How to change or disable it

```
// Lower niceness → less impact on the host, but agent jobs may be slower
{ "bash": { "nice": 5 } }

// Push as low as allowed
{ "bash": { "nice": 19 } }

// Disable priority wrapping entirely – commands run at normal priority
{ "bash": { "nice": null } }
```

Because **nice** is not merged across config layers, set it in the global config if you want a consistent value everywhere.

The **/bash** slash commands

The TUI exposes interactive management of the allowlist/denylist via **/bash**:

```
/bash add allow <cmd>           # add a command prefix to the allowlist
/bash add deny <pattern>         # add a deny pattern
/bash remove allow <cmd>         # remove an allowlist entry
/bash remove deny <pattern>      # remove a deny pattern
/bash show                       # display current allowlist / denylist
/bash help
```

Each **add/remove** accepts an optional **--global** flag to target the global config (`~/.config/ragent/agent.json`) instead of the project config (`.ragent/agent.json`). Changes persist to disk immediately and update the in-memory snapshot used by the tool.

Runtime behaviour

Timeout

The default timeout for a shell command is **120 seconds** (`DEFAULT_TIMEOUT_SECS`). It can be overridden per-call via the **timeout** parameter on the tool. On timeout the tool returns **"Command timed out after N seconds"** with **timed_out: true** metadata.

Process limits

Before spawning, the tool acquires a process permit from a semaphore that caps **concurrent shell processes at 16** (`crates/ragent-types/src/resource.rs`). Combined with a separate cap of 5 concurrent tools, this prevents an agent from forking runaway process trees.

kill_on_drop

Every process builder sets **kill_on_drop(true)**. This is critical: if a command times out or the future is dropped, the child process (and its descendants) are **killed** rather than being orphaned and left spinning at 100% CPU. This was a real bug fixed in v1.0.59 — repeated bash timeouts previously accumulated orphaned processes. The flag is now present at every spawn site (syntax check, all three execute branches, and all background-shell paths).

Output truncation

Output is truncated to the **first 15,000 + last 15,000 characters** (`MAX_OUTPUT` = 31,000) with UTF-8 boundary alignment and an **... lines omitted ...** separator, so very long command output never blows up the agent context.

Persistent shell state & working directory

The tool maintains a per-session state file that records the current working directory. Commands can **cd** within the sandbox and ragent tracks the change (publishing **Event::ShellCwdChanged** on the event bus) so subsequent commands run from the updated directory.

Sudo askpass broker

For the rare legitimate `sudo` case, `ragent` starts an `AskPassBroker` that routes the `SUDO_ASKPASS` prompt through the interactive question dialog instead of hanging on a terminal password prompt.

YOLO mode

YOLO mode (`crates/ragent-config/src/yolo.rs`) is an escape hatch for trusted, single-user environments. It bypasses **Layers 2, 3, 6, and 7** (banned commands, denied commands/patterns, user denylist, and obfuscation detection). Layers 1, 4, and 5 (directory-escape and syntax validation) remain active — YOLO does **not** let the agent escape its working directory or run syntactically-invalid commands.

Enable it with `/yolo`, `Alt+Y`, or by setting `"yolo": true` in config. Once enabled in either the global or project config it stays enabled (OR merge semantics).

Platform notes

- **Linux:** full treatment — `nice -n 10` and `ionice -c 3`.
- **macOS / other POSIX:** `nice -n 10` only (no `ionice`).
- **Windows (Git Bash / PowerShell):** no priority prefix is applied. The reason is documented in the source: Git Bash's `nice/ionice` may not be standard utilities, and the shell executable may not support them. Heavy commands on Windows therefore run at normal priority.
- **PowerShell:** syntax pre-validation (Layer 5) is skipped; the runtime parser inside the wrapper handles it. All other layers apply.

Reference

Files

Path	Role
<code>crates/ragent-tools-core/src/bash.rs</code>	The <code>BashTool</code> implementation
<code>crates/ragent-tools-core/src/bg.rs</code>	Background shell integration
<code>crates/ragent-config/src/config.rs</code>	<code>BashConfig</code> struct and defaults
<code>crates/ragent-config/src/bash_lists.rs</code>	Runtime allowlist/denylist/nice snapshot
<code>crates/ragent-config/src/yolo.rs</code>	YOLO mode
<code>crates/ragent-tui/src/app/slash.rs</code>	<code>/bash</code> slash commands
<code>crates/ragent-tui/src/app/init.rs</code>	<code>load_from_config()</code> at startup
<code>crates/ragent-types/src/resource.rs</code>	Process/tool semaphores (16 processes, 5 tools)

Config JSON keys (summary)

Key	Type	Default	Meaning
<code>bash.allowlist</code>	<code>string[]</code>	<code>[]</code>	Command prefixes that bypass the banned-command check
<code>bash.denylist</code>	<code>string[]</code>	<code>[]</code>	Substring patterns that always reject a command
<code>bash.nice</code>	<code>integer/null</code>	<code>10</code>	CPU niceness for all shell commands (<code>null</code> = normal priority)
<code>yolo</code>	<code>bool</code>	<code>false</code>	Master switch that bypasses security layers 2, 3, 6, 7

Examples

Tune priority to be friendlier to the host:

```
{
  "bash": {
    "nice": 15
  }
}
```

Re-enable `curl` (without YOLO) but keep it out of `rm` destructive forms:

```
{
  "bash": {
    "allowlist": ["curl", "wget"],
    "denylist": ["rm -rf /", "git push --force"]
  }
}
```

Run commands at normal priority (disable nice/ionice wrapping):

```
{
  "bash": {
    "nice": null
  }
}
```

This document describes the bash tool as implemented across ragent v1.0.59. The priority management (`nice -n 10 / ionice -c 3`) is active on Linux by default and is applied to both foreground and background shell commands.